Investigating and Combining Approaches for Data-Efficient Model-based RL

Jakub Bednarz JB406103@STUDENTS.MIMUW.EDU.PL
University of Warsaw
ul. Banacha 2
02-097 Warszawa

1 Introduction

Reinforcement Learning (RL), a research field dedicated to developing intelligent autonomous agents, capable of learning to do tasks in various environments, has achieved remarkable progress in recent times. Through the use of deep learning and artificial neural networks, algorithms playing video games at a super-human level or controlling robots without any prior knowledge have been developed.

However, most of the current state-of-the-art techniques have a significant drawback: they require excessive amounts of data (which, in this setting, we regard as the total time spent interacting with the environment) in order to achieve such performance. For example, standard benchmark on a collection of Atari video games assumes a total "budget" of 200 million game frames, equivalent to a human playing for $\approx$ 40 days without rest. This shortcoming greatly limits real-world deployment of such methods, as without a simulator of the target environment learning the proper behavior is impossible.

In order to remedy this problem, many approaches designed with *sample efficiency*, being the performance obtained with a given (small) number of interactions with the environment, in mind, have been proposed, such as:

- Improving sample efficiency of existing algorithms, without any major architectural changes, by carefully tuning the number of optimization steps per single environment step.
- Learning a model of the environment, with which one can either simulate environment interactions for the model-free method, or use the model in a more sophisticated way, e.g. through a planning algorithm.
- Better architectures of the neural networks comprising the agents.

All these have resulted in marked improvements in terms of sample efficiency. One may observe, however, that such methods have been developed independent of each other, and in many cases are orthogonal, meaning it is possible to envision a single RL agent combining many such strategies and modifications, hopefully capable of reaching even higher performance than standalone algorithms.

With this in mind, the goal of this thesis is to answer following questions:

1. *What are the existing methods for improving data efficiency of RL agents?*

2. *How to approach the design of a single algorithm combining these methods?*

3. *What are the synergies and interferences between these modifications?* And, relatedly, *what (if any) improvement in terms of performance can we expect from such an agent?*

The rest of this thesis is structured as follows: (...)

2 Background

2.1 Reinforcement Learning

Reinforcement Learning is a field of artificial intelligence and machine learning concerned with constructing intelligent agents capable of learning what actions to take in an environment so as to maximize a scalar reward signal.

Formally, the environment can be represented by a Markov decision chain (MDP) in the form of a tuple $\langle \mathcal{S}, \mathcal{A}, p \rangle$ consisting of the state space $\mathcal{S}$, action space $\mathcal{A}$ and transition probability $p(s_{t+1}, r_t \mid s_t, a_t)$, where $r_t \in \mathbb{R}$ denotes the reward granted at that step. Additionally, in the case of partially observable environments, we have an observation probability $p(o_t \mid s_t)$. The agent, starting from an initial observation o_1, acts according to a probability distribution $\pi(a_t \mid o_{\leq t})$ conditioned on previous observations until it reaches a *terminal* state s_T. The goal of reinforcement learning and the RL agent is to maximize the expected sum of rewards obtained along the trajectory $\mathbb{E}\left\{\sum_{t=1}^{T-1} r_t\right\}$.

[A section about Q-learning and actor-critic methods?]

2.2 Deep RL

[A section about DQN, SAC, PPO and other such stuff?]

2.3 Benchmarks for sample-efficient RL

A number of setups for evaluating data-efficient agents have been established in prior work.

Atari-100k For discrete control, the *de facto* standard benchmark is Atari-100k (Ref: SimPLe paper), a subset of 26 games from the full Atari Learning Environment (ref the paper) benchmark, limited to 400×10^3 game frames, equivalent to 100×10^3 agent stops assuming the standard action repeat value of 4.

2.4 Data augmentation in RL

Drawing from the common practice of using data augmentation (e.g. image transformations like rotation, random shifts) in supervised learning to improve performance and reduce overfitting, DrQ develops an RL-specific augmentation schema: beyond simply applying random transforms to observations, we may also use the fact that Q and V functions should be invariant w.r.t. the transforms to smooth them out by averaging.

2.5 Self-supervised RL

Standard deep RL methods, such as DQN or SAC, only utilize the reward signal in order to learn useful state representations to be used to learn the policy. Some authors have

proposed employing techniques from the area of self-supervised/unsupervised learning in order to combat this inefficiency.

Broadly speaking, self-supervised learning consists of adding additional *pretext tasks*, which are solved not for their own sake, but rather to provide learning signal for the downstream representations. Many kinds of pretext tasks have been suggested:

- For visual input, one option is to exploit inherent structure of images, via rotation prediction, colorization (reconstructing the image from a grayscale version) or jigsaw (splitting an image into patches, randomizing the order and reconstructing the original image).

- Contrastive learning (CL) approaches, based on the instance discrimination task, construct such representations, that views (augmentations, crops etc.) of the same instance ("positive pairs") are pulled together, and views of different instances ("negative pairs") are pulled apart. Some CL methods do not use negative pairs altogether, relying on momentum, batch normalization and projection heads to avoid collapse of the representations.

- Generative approaches reconstruct input from noisy or corrupted versions. Autoencoders, variational autoencoders (VAEs) and denoising VAEs pass the (possibly augmented) input through a latent bottleneck in order to recover the original input. The corruption can also take form of masking patches of the input - this approach has proven especially suited to pre-training representations for NLP and Vision Transformers.

Similarly, self-supervised RL techniques attempt to incorporate pretext tasks into regular RL pipelines. Arguably the simplest way is to apply SSL to augment learning of observation embeddings. For example, CURL takes a base RL model (in this instance SAC) and uses MoCo to help train the observation encoder via an auxiliary CL loss term. Some architectures, however, design the pretext task to be more RL-specific. SPR, for instance, treats a future state and a state predicted by a transition model from the past state and the sequence of actions in between as a positive pair. Masked World Models use the paradigm of masking image patches and reconstructing them, with an important addition of also predicting rewards, which proves crucial for achieving high final performance.

2.6 Model-based RL

Whereas a model-free RL agent seeks only to learn the policy π, a model-based agent seeks also to model the probability, or simply allow sampling, of future states and rewards $\widetilde{s_{t+1}}, \widetilde{r_{t+1}}, \ldots, \widetilde{s_{t+H}}, \widetilde{r_{t+H}}$ given past observations $o_{\leq T}$ and actions to perform $a_t, \ldots, a_{t+H-1}$, where H denotes the imagination horizon.

2.6.1 Learning the model

SimPLE performs the transition in input (image) space - the UNet-like model receives input consisting of last K frames, as well as action conditioning, and is trained to output next frame. Stochasticity of the environment is modeled by encoding the input-output pair into a

discrete latent vector (intuitively representing which of the possible futures to expect) which is fed to the decoder; the distribution of these latent vectors is modeled autoregressively with an LSTM for use at inference time. The issue with the approach is that encoding and decoding images in such a fashion proves to be computationally expensive. Thus, other model often opt to perform modelling in the latent space. World Models, for example, uses a two-stage training recipe: visual inputs are first compressed into a latent vector using a Variational Autoencoder, whereafter a combination of an RNN and a "distribution head" model the transition probabilities in the latent space. In (PlaNet), the authors raise the issues surrounding modelling stochastic environments autoregressively and propose the use of both deterministic and stochastic recurrent networks in order to deal with stochasticity without inducing the decay of the information stored in the state vector. This approach has been further refined in the Dreamer series of models by e.g. using discrete latents or employing better techniques for pulling together prior and posterior state distributions.

Model states do not necessarily need to correspond to real states - in the case of *value-equivalent models*, the states and the transition function are optimized to approximate the values to be obtained after executing a sequence of actions. (Speak of MuZero &c?)

2.6.2 Using the model

Let us take a look now at how such a world model can be used in an RL agent. One class of methods uses the model for *planning*; such techniques are also known as *lookahead methods*. Here, planning refers to any decision process which actively uses the world model. For example, in its simplest incarnation, Model-Predictive Control (MPC), at every time step, optimizes a sequence of actions $a_t, \dots, a_{t+H-1}$ such that the expected total rewards $\mathbb{E}\left\{\sum_{k=0}^{H-1} r_{t+k}\right\}$ is maximized, and selects the first action a_t - at the next step, the procedure is performed again. The action sequence can be optimized using any zero-order method, like Monte Carlo (i.e. sampling the actions randomly and selecting the one with the best score), or through Cross-Entropy Method (CEM). A variation of this idea is to employ a Monte Carlo Search Tree in order to focus on more promising directions of exploration. In all cases, an auxiliary value function predictor can also be used in order to approximate the returns beyond the rollout horizon.

At the other end of the spectrum lay approaches which use model rollouts for learning the policy. Such rollouts can be used either as a data augmentation technique, or completely replace training on real data, instead learning behaviors entirely in the "imagination" via regular model-free RL algorithms.

2.7 Training recipes for data-efficient RL

A recent line of research has focused on achieving greater sample efficiency without any architectural changes - rather, the idea is to optimize the training recipe, which we define as the way optimization and environment interaction steps are organized throughout the training process. In some cases, as shown in (Data-Efficient Rainbow), it is sufficient to simply increase the frequency of optimization steps per environment step. At sufficiently high replay ratios, the final performance of the agents starts to decrease. In (Primacy Bias paper), the authors analyze this phenomenon, and suggest that the fundamental cause of this is *primacy bias*, whereby the neural networks overfit to early interactions, causing the

loss of ability to learn and generalize from more recent data. The proposed remedy is to periodically fully or partially reinitialize the networks used by the agent - this simple procedure is shown to significantly improve performance in low-sample regime.

2.8 Integrated architectures

The main idea of this paper is inspired and motivated by Rainbow architecture and associated paper. In it, the authors took a base off-policy model-free RL algorithm, DQN, applied an array of modifications developed by other authors (specifically: double Q-learning, dueling networks, prioritized sampling, noisy networks, distributional RL and multi-step TD targets), adapted to fit together in a unified architecture, and achieved substantial improvements in terms of final performance of the agent.

Our goal is to develop a framework for sample-efficient RL, in a similar fashion to Rainbow - taking a base architecture, an array of approaches developed to improve sample efficiency, and combine them together into a single architecture. To our knowledge, this has not yet been investigated.

3 Method

Let us now consider how these advancements can be integrated into a single unified, modular framework for constructing sample-efficient RL agents.

The use of world models in a number of state-of-the-art approaches for data-efficient RL informs our choice to design the architecture in a model-based fashion. Thus, the framework shall consist of a world model, the RL agent proper, along with a replay buffer.

In many cases, the advances in sample efficiency result from the development of better model or RL algorithm architectures *in isolation*. With this in mind, we wish to make the framework allow for possibly seamless swapping out of the model and RL modules. In order to do this, we need to examine how different model-based architectures structure the interaction between the model and the RL modules.

1. Dreamer series takes the approach of training a (possibly arbitrary) model-free RL algorithm purely with simulated experience. The model and the RL modules are essentially independent of each other.

2. One may mix real and simulated data for the purposes of training the RL agent. For example, in MBPO, the authors use an on-policy method, but augment the data with imaginary rollouts.

3. The agent module may not even require any training per se. For example, PETS and PlaNet use cross-entropy method (CEM) for selecting actions to perform by planning with the learned model.

4. Methods using value-equivalent models, such as MuZero or EfficientZero, train the model and the RL module jointly. In this instance, the interaction is more complex.

Our desiderata are best fit with a generic approach inspired by the Dreamer series. To be precise, we consider a following scheme:

1. The world model is independent of the RL module, and is optimized on samples from a replay buffer.

2. The RL module may use the model actively, and may require optimization using batches of on- or off-policy rollouts, either real or simulated, or a mixture of both (if the observation and model state spaces coincide.)

This formulation subsumes most of the algorithms we have considered so far, such as Dreamer, MBPO, PlaNet, SimPLe, or IRIS: the specific method can be replicated by adjusting the architectures of the model and of the RL module, or the form of the input data for the RL module optimization step. Data augmentation may, for example, be implemented by placing a layer between the replay data loader and the model.

Beyond the network architecture, a major component of the design of sample-efficient agents is the structure of the training loop, meaning the order and frequency in which environment interaction, model optimization and RL module optimization occur. Indeed, works such as Data-Efficient Rainbow or SR-SPR prove that ordinarily inefficient methods can be made vastly more data-efficient by refining the training process. We wish to extend these findings to the domain of model-based RL. This is, however, non-trivial, and requires further investigation.

First, there are purely technical difficulties. Let us, for example, consider scaling up the frequency of model and RL updates. As opposed to the model-free case, we are dealing with two separate learning processes, that of the model and of the RL module, resulting in two separate update frequencies to be tuned. If we were to search for the optimal configuration through a grid search, we would be faced with a quadratic number of training runs to perform, relative to the model-free case, which would quickly become infeasible. It is therefore imperative to reduce the extent of the hyperparameter sweep as much as feasible.

Next, insofar as the collapse of the performance for regular model-free RL agents, when increasing the update-to-data frequency, is attributed to plasticity loss, it is not immediately clear whether model-based agents undergo the same process, whether the model and the RL module suffer from the primacy bias or perhaps only one of them does, and whether interventions proposed in the literature prove beneficial here as well. If such interventions require tuning, as is for example the case for periodic resets, then we possibly encounter another computationally expensive grid search.

Because the optimal training loop parameters are dependent on the specific model and RL modules used, we split our investigation into two parts. First, we shall use a baseline model and RL setup to examine the impact of various design choices in the training loop optimization, with the view of developing architecture-agnostic solution for tuning it. Thereafter, we will look into architectural ablations, to investigate their impact on the performance of the agent.

4 Training recipe optimization

4.1 Base architecture

We choose DreamerV2 architecture as the baseline for the purposes of optimizing the training recipe. The choice is driven by a number of factors:

1. We wish to start with a non-sample-efficient setup, and (hopefully) make it more data-efficient.
2. We want to focus on the visual domains, such as Atari-100k.
3. The model and the RL module are, asymptotically, capable of reaching high performance, meaning we will not be bottlenecked by the insufficiency in the model/RL modules themselves.

The agent consists of two modules:

- The world model – an observation encoder and a Recurrent State Space Model (RSSM) for modeling stochastic latent dynamics.
- The RL module – an actor-critic algorithm, optimized either via backpropagation on the values themselves, or through the use of policy gradients.

The training loop is structured as depicted in Algorithm 1.

Algorithm 1 Dreamer training recipe

1: Gather P env samples with an agent acting randomly, and put them into a replay buffer
2: **while** Total number of env steps $< T$ **do**
3: Optimize the world model using batch of real rollouts $(o_{1:L}, a_{1:L-1}, r_{1:L-1}, \gamma)$ drawn from the replay buffer.
4: Optimize the RL agent using the batch of dream data $(s_{1:H}, a_{1:H-1}, \widetilde{r}_{1:H-1}, \widetilde{\gamma}_{1:H})$, generated by starting from the states obtained through an inference process on real-world rollouts, and imagining next $H-1$ states with the world model.
5: Perform $E = 64$ environment steps using the behavior policy, and store them in the replay buffer.
6: **end while**

For more details, we refer the reader to the source paper. For our purposes, though, we wish to treat these modules as "black boxes".

4.2 Experimental setup

We are interested in evaluating performance on the Atari-100k benchmark. However, performing training runs on the entire set of 26 games and with a full range of RNG seeds for every modification is computationally infeasible. Thus, we will use a subset of these games and a reduced number of seeds as a proxy. To be specific, we shall limit ourselves to a choice of 3 environments and 5 RNG seeds. In order to properly gauge the effect of changes on the full set, we want to construct a subset of Atari-100k with following properties:

1. Since we are interested in *speeding up* the algorithm, we ought to select such environments, in which the method makes consistent progress in a given extended interaction budget. For example, given a maximum speedup target of $15\times$, we would focus on the horizon of $15 \cdot (400 \times 10^3) = 6 \times 10^6$ environment steps. If the method doesn't improve much beyond the performance of a random agent, or solves the task immediately, gauging the speedup factor becomes impossible.

2. The variance of the results with respect to different RNG seeds used should be minimized, so as to be able to derive the expected score with a limited number of RNG seeds with any degree of accuracy.

3. The subset should be as representative of the performance on the whole subset as possible.

First, let us filter out the environments which do not satisfy first two criteria. In the absence of standardized metrics which would inform our choice quantitatively, we resort to visual inspection of the performance curves for each environment in the first 6×10^6 steps. The relevant data can be found in Figure 1. Based on it, we select following games for further evaluation: `Amidar`, `Assault`, `Asterix`, `CrazyClimber`, `JamesBond`, `MsPacman` and `Pong`.

Next, we shall select out of this subset 3 games, which are most predictive of the performance on the complete 26-game benchmark. We follow the approach of (Atari-5), and use the source code provided by the authors to arrive at the final result. The three environments selected by the algorithm are: `Assault`, `CrazyClimber` and `MsPacman`.

We use the same environment settings (frame skip values, time limit etc.) as the original DreamerV2. (See Appendix for more details?)

The original implementation of DreamerV2 is written in Tensorflow - we have ported the relevant source code (mainly the implementation of the world model and the actor critic) to Pytorch, resulting in slight changes in overall performance. It remains, however, comparable: Figure 2 depicts the validation score curves for both the original implementation, and the Pytorch version.

4.3 Naïve speedup test

We begin with an experiment involving a straightforward increase of the frequency of optimization steps, without any further changes. The goals are as follows: to establish a baseline for further analysis, to find the update-to-data regime where the agent performance degrades, and to investigate what happens at that point with the networks in question.

Setup Base DreamerV2 training recipe begins with a prefill of $P = 200 \times 10^3$ and continues until the total step count is $T = 200 \times 10^6$. The latter loop consists of $E = 64$ env steps (at the frame skip value of 4, this corresponds to 16 agent steps) and a single optimization step.

We replace T by 400×10^3 to match Atari-100k benchmark environment budget, and P with 20×10^3. Then, we perform tests with varying values of $E \in \{64, 32, 16, 8, 4, 2\}$, for a set of 3 selected environments and 5 RNG seeds.

Beyond that, for testing purposes, we also add an online train-validation split mechanism: each episode in the replay buffer is assigned either to the training set or the validation set – to be precise, every $(1/p)$-th episode is assigned to the validation set, where $p = 0.15$.

Results Let us first analyze final test performance, depending on the data-to-update ratio E, for each environment (Figure 3). The mean final score is maximized for a given region of the values of E, beyond which it decreases, presumably either due to under- or overoptimization. For each task, the optimal value is different: $2 - 4$ for `CrazyClimber`, ≈ 8 for `Assault` and a wide range for `MsPacman`.

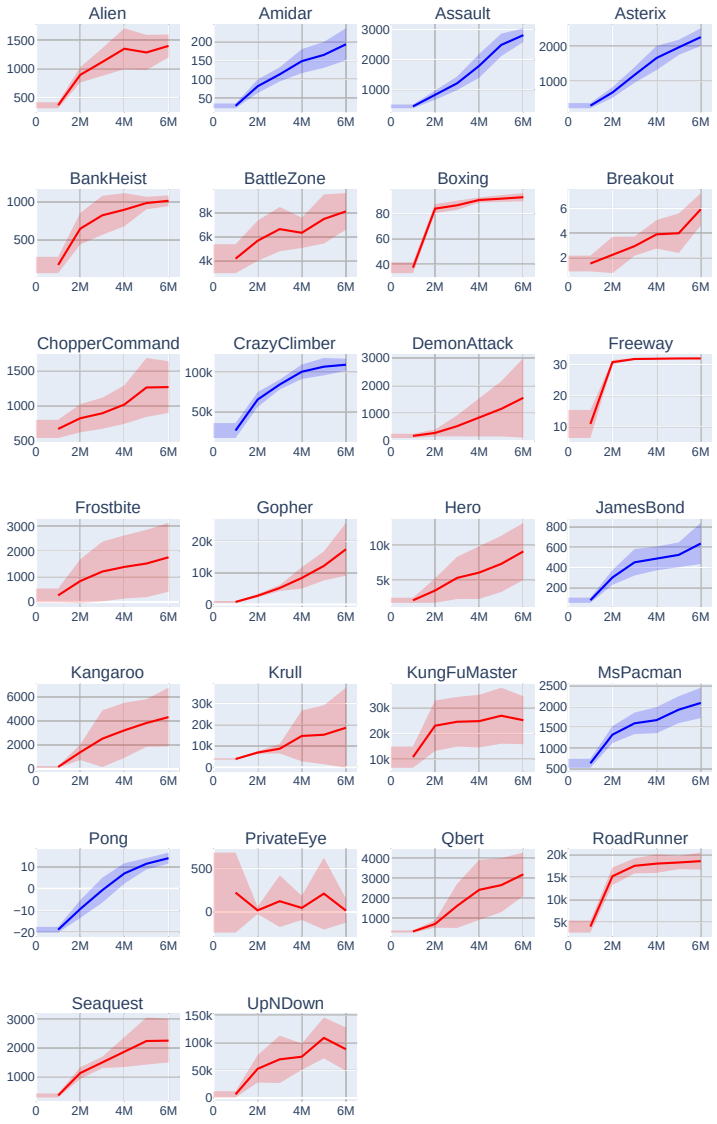

Figure 1: Performance curves for Atari-100k, with default settings. The values have been provided by the authors of the DreamerV2 paper. The x-axis represents the number of environment steps passed. The y-axis represents the validation score. Blue curves indicate environments, which we have selected for further evaluation.

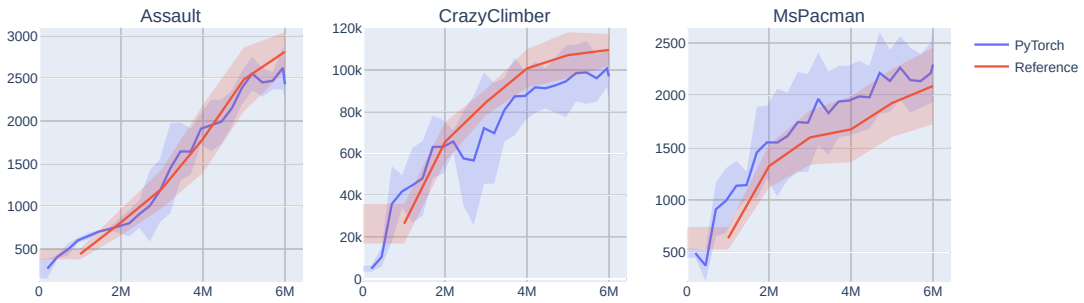

Figure 2: Comparison of the validation scores for Tensorflow and Pytorch versions. *Note:* The scores for the reference (Tensorflow) version have been supplied by the authors, and were not verified independently.

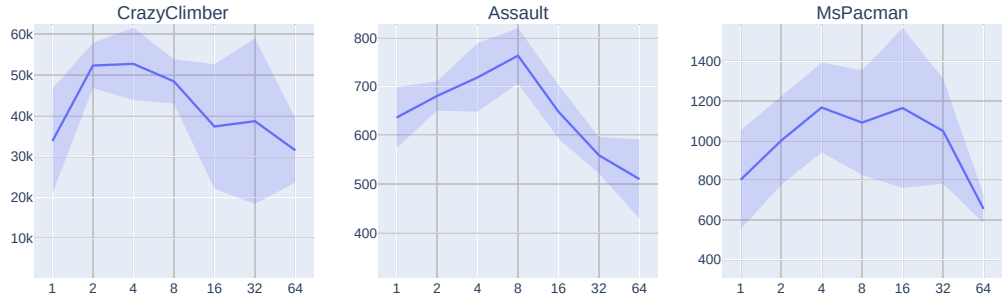

Figure 3: Final test episode returns for each of the 3 tested environments, relative to the data-to-update ratio E. The line indicates the mean across the RNG seeds, and the shaded area indicates the standard deviation. *Note*: The x-axis is logarithmic.

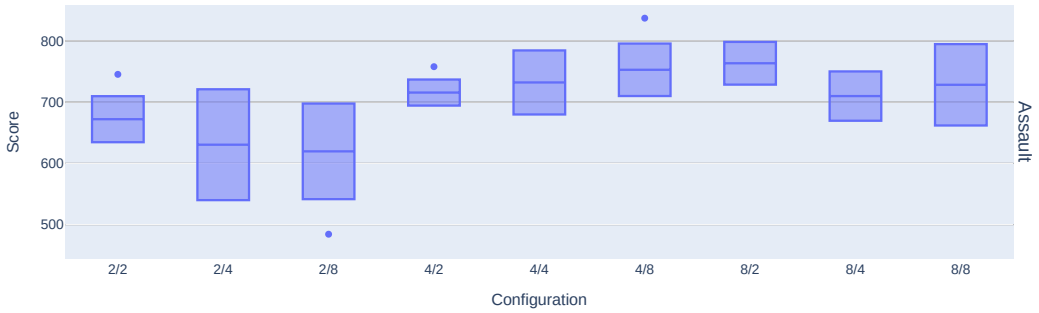

Figure 4: Final validation score for each model/RL update ratio configuration. The x-axis represents each run, marked by a tag in the form of '$K_{\mathrm{WM}}/K_{\mathrm{RL}}$'. The points represent the scores for each seed, with the corresponding box plot to the right.

4.4 Decoupling model and RL optimization

As noted previously, the optimal training schedules for the model and the RL module need not coincide - we may, for example, find that the model is overfitting, yet the RL head remains underoptimized. The next experiment aims to investigate the empirical effects of decoupling model and RL update frequencies.

Setup We adjust the code, so that the model is optimized once every K_{WM} environment steps, and the RL module is optimized once every K_{RL} environment steps. A grid search is performed for `Assault` environment [Extend to all three, if there's enough compute time], with ratios $K_{\mathrm{WM}}, K_{\mathrm{RL}} \in \{2, 4, 8\}$, corresponding to the observed regime, where the performance starts decreasing.

Results Figure 4 shows the agent performance depending on the configuration of the ratios. Both the model and the RL ratios play a significant role in determining the final score, though no clear pattern can be determined - for example, increasing RL update ratio does not necessarily improve the performance.

4.5 Investigating the learning dynamics of the agents

So far, the experiments suggest that achieving optimal or near-optimal performance may require, at the very least, a hyperparameter sweep over *both* model and RL update frequencies. Given the increase in the number of training runs this would require, we will now investigate methods to auto-select at least some of these parameters. In order to achieve this, we will start with an examination of the various metrics thoughout the run, and see whether they are correlated with the final score. The idea is that if we are able to control them throughout the optimization process, we may be able to guide them in such a way so as to achieve higher final performance.

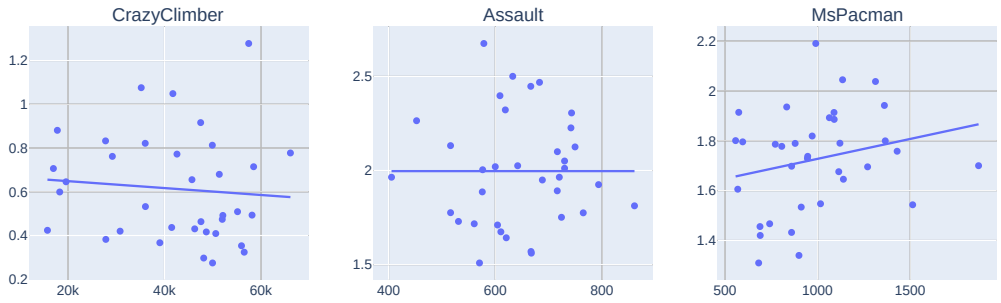

Figure 5: Relationship between final validation score and final model validation loss. Each point represents a single training run. The x-axis represents the agent performance. The y-axis represents the value of the model validation loss at the end of the training.

Since model learning is a supervised learning task, arguably the most imporant metric to monitor is validation loss. To be precise, we compute mean loss value over the batches of data drawn from the validation episodes. We will start with the aggregate results - the relationship between the final validation loss and the final agent performance, for the equal-ratio experiment described in subsection 4.3. As we can see in Figure 5, it would appear, somewhat surprisingly, that the value, for individual trials, is not meaningfully correlated with the final score achieved by the agent. A closer inspection of the validation loss curves (Figure 6) reveals, that the agents do not, in general, exhibit classical overfitting behavior, in the sense that the validation loss curves plateau at worst, and in general the loss value keeps decreasing. The learning process nevertheless degrades, as the final values of the validation loss keep increasing as the ratio of the optimization and environment steps increases. The ratio values, for which the final loss value is the smallest, do not correspond to the ones which achieve the highest performance.

To further examine the behavior of the agent throughout the optimization process, let us take a look at the *training* loss curves. An aggregate comparison (Figure 7) reveals a degree of correlation between the agent performance scores and the training loss values. The loss curves (Figure 8) are of altogether different nature, compared to the validation loss curves. We can see a clear monotonic relationship between the final loss value achieved and the update frequency, though, as before, the better values do not lead to improvements in terms of the quality of the agents. Perhaps more interestingly, at very high update frequencies (i.e. small data-to-update ratios), the optimization process seems to stall out completely. For the value of $K = 1$, for example, in all three environments tested, near-"optimal" value is reached after the first 100×10^3 environment steps, whereafter it either improves marginally, or even increases. [Well, what to make of *that*?]

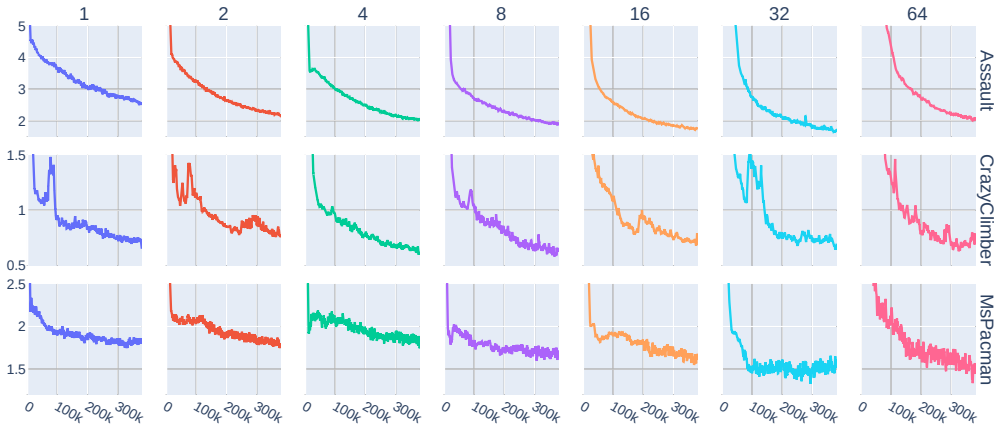

Figure 6: Averaged model validation loss curves for each configuration of the environment and the update ratio used. Rows represent different environments, and the columns the different update ratios used. The x-axis corresponds to the number of environment steps taken, and the y-axes represent loss values. The ranges of the y-axes are different for each environment.

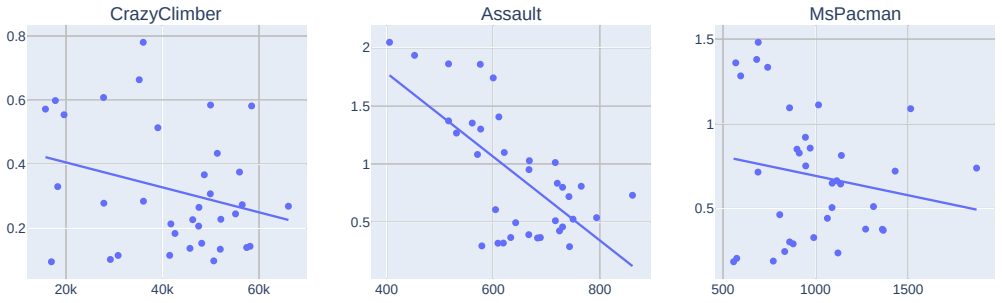

Figure 7: Relationship between final validation score and final model training loss. Each point represents a single training run. The x-axis represents the agent performance. The y-axis represents the value of the model training loss at the end of the training.

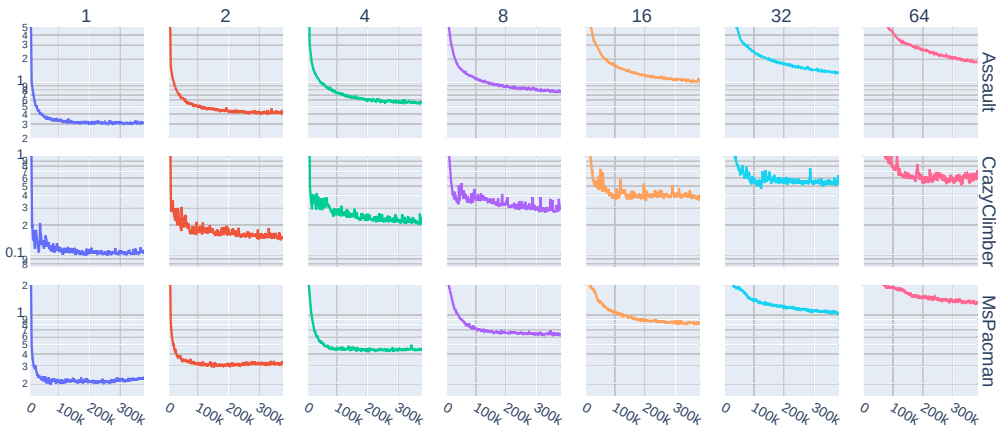

Figure 8: Averaged model training loss curves for each configuration of the environment and the update ratio used. Rows represent different environments, and the columns the different update ratios used. The x-axis corresponds to the number of environment steps taken, and the y-axes represent loss values. The ranges of the y-axes are different for each environment, and are **logarithmic** due to the wide range of values.

Analysis of the behavior of the RL component is less straightforward, as there is no analogue of under- or overfitting of supervised models in the case of reinforcement learning. There are, however, a couple of phenomena, which can cause performance collapse, such as:

1. Value overestimation: Since most RL algorithms compute value estimates via bootstrapping, it is possible for bad initial value estimates to be amplified by the optimization process.

2. Model exploitation: In the context of model-based RL, and specifically the methods based on Dreamer architecture, since all the data given to the RL module is imagined by the model, inaccuracies thereof can cause the agent to learn wrong policies which perform well in the model-induced MDP.

3. Policy collapse: for actor-critic methods, without any regularization, the policy will tend to rapidly concentrate on the actions with the highest advantage estimates, which in turn would hurt exploration and prevent the agent from learning more optimal behaviors.

To investigate these, we will first take a look at the value curves. [?]

4.6 Auto-tuning update frequencies

It would therefore appear that these metrics cannot be directly used for the purposes of guiding the optimization process. There exist, however, works doing precisely that, which demands a further inquiry.

We will specifically do a closer examination of (Dynamic update ratios paper). The approach taken by the authors is to check validation loss throughout the training process, and to decrease the update frequency if the loss value has degraded, and increase it if it has improved. To start off, we will replicate the experiment, albeit with some notable changes:

1. The authors propose using image reconstruction loss as the validation metric. We would rather have the method apply to models other than DreamerV2's specifically, however, so will use the total loss value.

2. A validation set is constructed differently: 3×10^3 transitions are collected every 100×10^3 steps, which corresponds to the flat 12% of the transitions being reserved for the validation set, whereas we reserve 15% of the *episodes* instead. Thus, the training set contains transitions which immediately precede or succeed the ones in the validation set - whether it represents a form of data contamination is unclear.

The results can be found in Figure 9, along with the baseline results for the sake of comparsion. It would appear that the strategy is, in general, effective, though not universal - for example, the scores on `Assault` do not quite reach the performance peak.

Given that the idea behind this approach is to minimize model overfitting by monitoring validation loss, our next step is to look at the validation loss. The results (Figure 10) do not seem to imply that the method has a consistent positive effect on the model validation loss. In fact, if we compare validation loss curves and the current ratio values (Figure 11), the

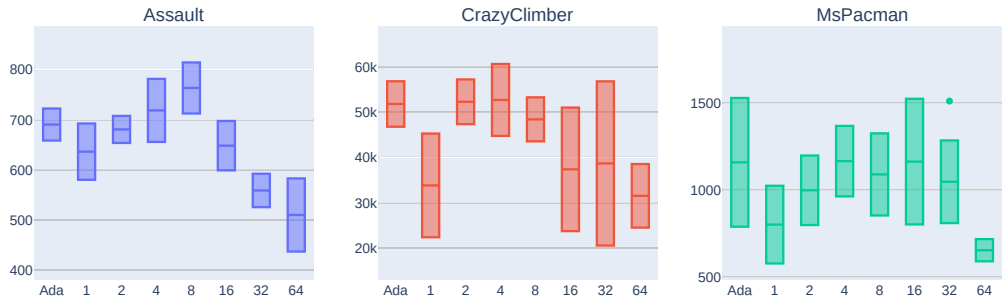

Figure 9:

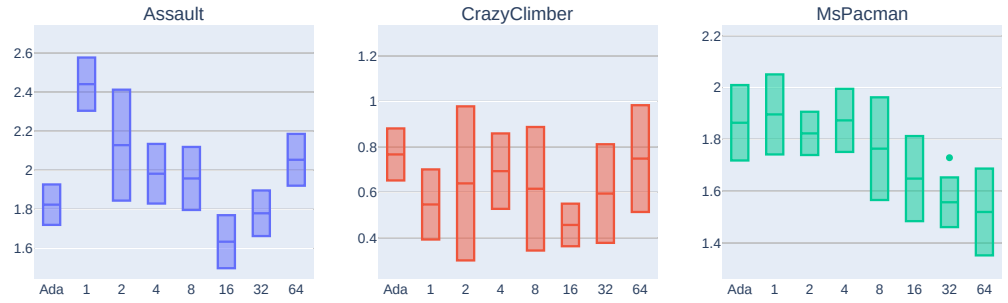

Figure 10:

relationship is not precisely what we would expect. As noted previously, the idea behind the method is to increase update ratio as long as the validation loss decreases, and vice versa. In short time scales, specifically the 2×10^3 environment steps between each ratio update, this does occur, but across longer time spans, the coupling breaks. For example, for `MsPacman` and the RNG seed of 1, the validation loss decreases up until the timestep of 200×10^3, yet the ratio keeps increasing. After the jump, presumably due to the influence of a new validation episode in the buffer, the loss value starts decreasing, and the ratio decreases.

4.6.1 Decoupled adaptive ratio system

It appears, then, that the theoretical underpinnings behind the effectiveness of this approach are not quite as clear as one would hope. Despite this, we believe one can nevertheless improve upon this method in a principled fashion. Drawing on the results concerning the

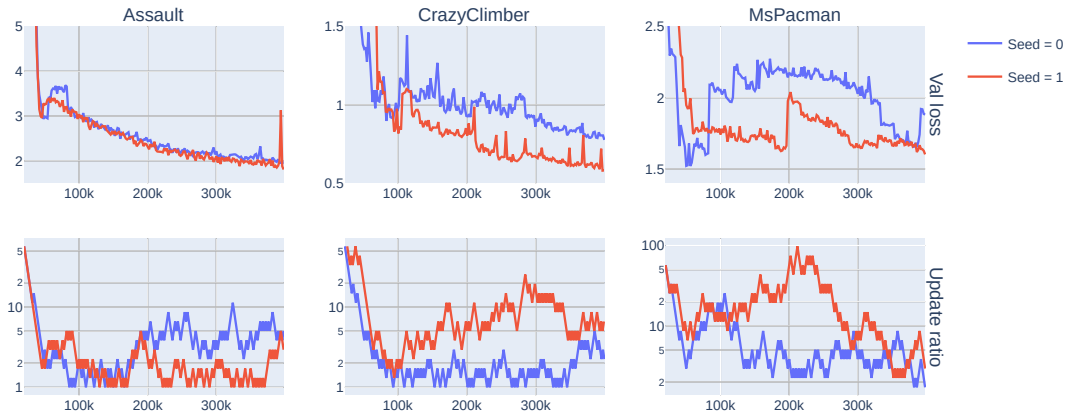

Figure 11:

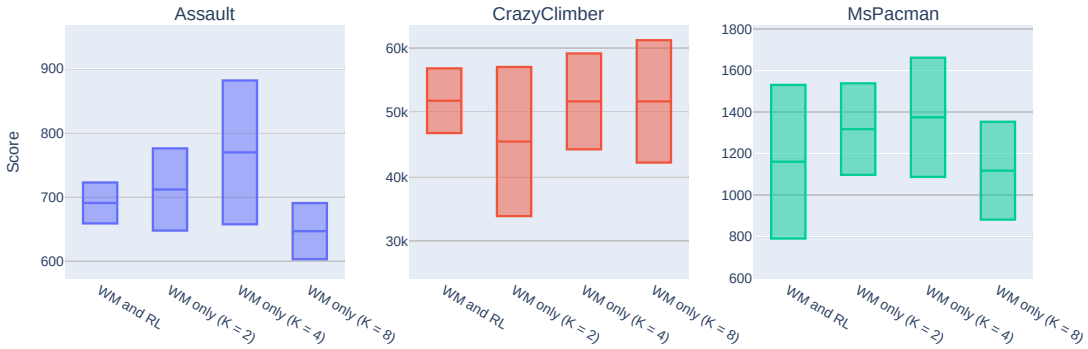

Figure 12:

decoupling of the optimization rates for the model and the RL modules, we propose a similar decoupling in the adaptive regime. In particular, we will make only the model update ratio adaptive, and keep the RL update ratio fixed at a pre-defined value. The results (Figure 12) indicate, that the decoupling has a positive impact on the agent performance.

5 Architectural changes

5.1 Data augmentation

Incorporating data augmentation into an RL framework can be done in non-trivial ways. DrQ, for example, combines transforming the input, averaging the Q functions across mul-

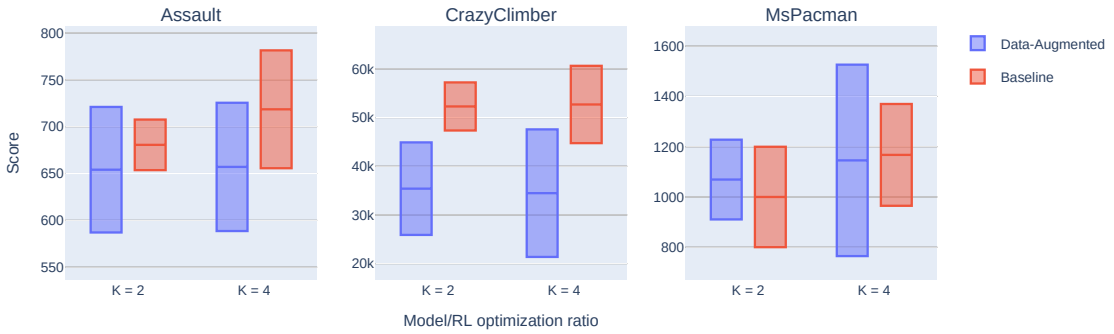

Figure 13:

tiple augmented samples, and averaging the Q targets. In the context of model-based RL, one could develop similar schemes for both the world model and the RL modules, e.g. by averaging posterior state distributions over multiple image augmentations. This, however, would require changing the implementation of these modules, whereas our goal is to make the framework as modular as possible. Therefore, we opt to only augment the input, and forego the averaging of various quantities in question.

We follow the original paper in choosing random shifts as the image transformation to apply. The augmentations will be applied to both images in the batch for the model optimization, and the batch used to generate the initial states for the imagined rollouts. As far as the training recipe is concerned, we will start with a fixed-ratio setup in order to compare the data-augmented recipe with a non-data-augmented baseline.

Results The comparison of the test episode returns for the two variants (Figure 13) shows, that, with the base settings, the addition of data augmentation *degrades* performance, excepting perhaps `MsPacman`. As the main idea behind data augmentation is the reduction of model overfitting, we will next investigate the training and the validation loss curves for the model. The analysis of the final validation loss for each variant (Figure 14) reveals, that not only is it significantly higher compared to the baseline variant, but it also does not improve as the update frequency increases (equivalently, K decreases).

If we take a look at the model training loss curves (Figure 15), we may observe a following phenomenon: for `Assault` and `CrazyClimber`, the addition of random shifts appears to slow down, or even stall out the training process, as shown by the slower rate of decrease (in log-space). For `MsPacman`, however, the reverse is true: in the original experiment, training loss stops improving at a certain point, whereas in the data-augmented scenario, we observe a consistent decrease. [What further conclusions to draw?]

5.2 Choice of the RL algorithm choice

So far, all the experiments have used the default RL module of DreamerV2, namely a variant of Advantage Actor-Critic (A2C). In this section, we shall investigate the effects

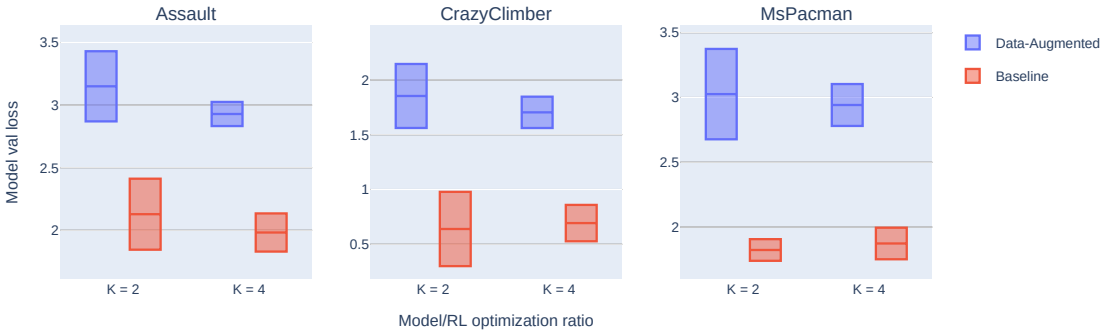

Figure 14:

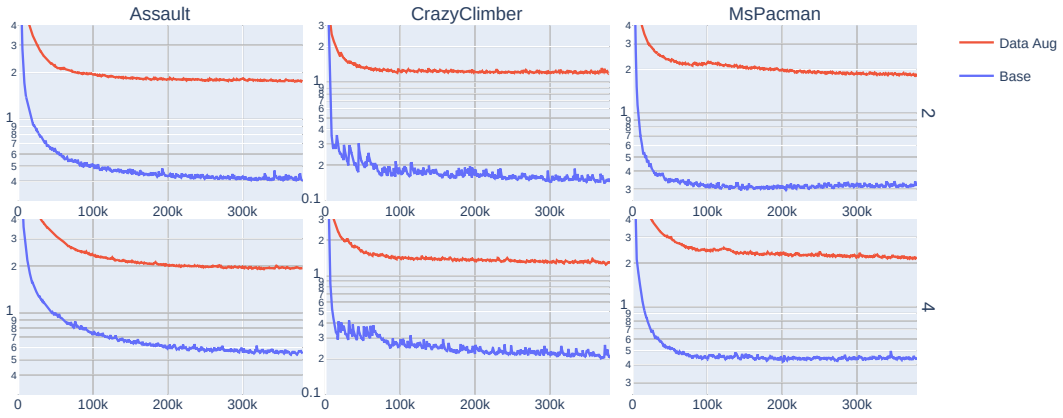

Figure 15:

of replacing it, whilst keeping the model component intact. Beyond hopefully achieving improvement in terms of agent performance, our objective is to elucidate the design choices when incorporating model-free RL algorithms into a model-based setting.

The RL algorithms we wish to test are Soft Actor-Critic (SAC) and Proximal Policy Optimization (PPO). The selection is motivated by the a number of factors:

1. In the first order, we would like to conduct an incremental improvement over the original A2C algorithm. (...)

2. The data given to the RL module consists of imagined rollouts, with the actions being selected by the same policy that is being optimized. We should therefore be able to use on-policy algorithms, which are, in principle, more stable and performant, at the expense of data efficiency, due to having to interact with the environment at every optimization step. (Citation!) We would like to assess whether it holds in the model-based setting. We select PPO specifically as the reference "off-the-shelf" on-policy RL algorithm.

We follow CleanRL's implementations of SAC and PPO for Atari, with a couple of important changes:

1. Since the "observations" given to the model are in fact world model states, we change the default convolutional encoders to the feedforward networks used in the A2C implementation.

2. Imaginary sequences generated by the model are unlike the regular sequences drawn from e.g. the replay buffer. Because we do not know *a priori* whether a sampled state is terminal or not - we may only use the terminality predictor of the model, which would return the probability that a state is terminal or not - we need to account for a degree of "fuzziness" (in the sense of fuzzy logic) of the latent states. The alternative could be to stop a particular rollout during the generation process, when the probability exceeds a given threshold. In that instance, however, we would have to deal with batches of sequences of non-uniform length, which would likewise require changes to the implementation of the RL algorithm, and would likely be computationally inefficient, since we could not perform batched operations easily. We opt, therefore, for the former approach; this is also the approach taken in the DreamerV2's implementation of A2C.

3. A2C uses generalized advantage estimation to compute the targets for the value functions, whereas CleanRL's SAC implementation uses a one-step bootstrap. To ensure fair comparison, we implement a "soft" version of the generalized advantage estimation in SAC. [See Appendix for more details?]

To start off, let us test a configuration of SAC analogous to the default one for A2C - namely, with the same entropy coefficient, actor/critic optimizers, sequence length and batch size etc. The two major differences are: the use of the soft variants for the value and Q functions, and the use of two Q functions and taking the minimum of them.